

Problem Set 7 – Autonomous Payload Packet Parser

Kshitiz Reddy
M.Tech AIML, School of AI, Bennett University

Source Code

```
1  # The / in a signature marks everything before it as positional-only -
2  # callers can't use those parameter names as keywords, only position.
3  # The * marks everything after it as keyword-only - callers must name
4  # those parameters, position won't work. Together they let an author
5  # lock down exactly how each parameter can be supplied, which matters
6  # for API stability (positional-only names can be renamed later without
7  # breaking callers) and safety (keyword-only args can't be swapped by
8  # accident if two same-typed parameters sit next to each other).
9  #
10 # Slicing inside the comprehension (raw_packet[1][:]) is enough to avoid
11 # aliasing because a slice always builds a brand-new list object holding
12 # the same values - it never just points back at the original list. The
13 # comprehension itself also builds a new list for its result, so nothing
14 # in the return value shares identity with raw_packet[1].
15 #
16 # Author: Kshitiz
17
18
19 def parse_payload(raw_packet: list, delimiter: str, /, *,
20                  correction_offset: int = 0) -> tuple:
21     status_tokens = raw_packet[2].split(delimiter)
22     corrected_sensors = [
23         value if "ERROR" in status_tokens else value + correction_offset
24         for value in raw_packet[1][:]
25     ]
26     return raw_packet[0], corrected_sensors
27
28
29 # --- Test 1: normal correction ---
30 packet1 = [501, [12.5, 13.0, 11.8], "NOMINAL|CALIBRATED"]
31 original1 = packet1[1][:]
32 result1 = parse_payload(packet1, "|", correction_offset=2)
33 print(f"Test 1 -> {result1[0]} {result1[1]}")
34 assert packet1[1] == original1
35 assert id(result1[1]) != id(packet1[1])
36 print("Test 1 side-effect assertions passed.")
37
38 # --- Test 2: ERROR code suppresses the correction ---
39 packet2 = [502, [9.0, 8.5], "ERROR|SENSOR_FAULT"]
40 original2 = packet2[1][:]
41 result2 = parse_payload(packet2, "|", correction_offset=5)
42 print(f"Test 2 -> {result2[0]} {result2[1]}")
43 assert packet2[1] == original2
44 assert id(result2[1]) != id(packet2[1])
45 print("Test 2 side-effect assertions passed.")
46
47 # --- Test 3: default offset ---
48 packet3 = [503, [1.0, 2.0], "NOMINAL"]
49 result3 = parse_payload(packet3, "|")
50 print(f"Test 3 (default offset) -> {result3[0]} {result3[1]}")
51
```

```

52 # --- Test 4: illegal keyword call on positional-only params ---
53 try:
54     parse_payload(raw_packet=packet1, delimiter="|")
55 except TypeError as e:
56     print(f"Test 4 passed. TypeError raised as expected: {e}")
57
58 # --- Test 5: illegal positional call on keyword-only param ---
59 try:
60     parse_payload(packet1, "|", 2)
61 except TypeError as e:
62     print(f"Test 5 passed. TypeError raised as expected: {e}")
63
64 # Submission Note: Test 4 called parse_payload with raw_packet and
65 # delimiter as keywords, which isn't allowed since both sit before the
66 # / in the signature. Python raised "parse_payload() got some
67 # positional-only arguments passed as keyword arguments: 'raw_packet,
68 # delimiter'". Test 5 tried to pass correction_offset positionally as
69 # a third argument, which isn't allowed since it sits after the *.
70 # Python raised "parse_payload() takes 2 positional arguments but 3
71 # were given". Both matched what the assignment predicted.

```

Trace Log

```

1 Test 1 -> 501 [14.5, 15.0, 13.8]
2 Test 1 side-effect assertions passed.
3 Test 2 -> 502 [9.0, 8.5]
4 Test 2 side-effect assertions passed.
5 Test 3 (default offset) -> 503 [1.0, 2.0]
6 Test 4 passed. TypeError raised as expected: parse_payload() got some
   positional-only arguments passed as keyword arguments: 'raw_packet,
   delimiter'
7 Test 5 passed. TypeError raised as expected: parse_payload() takes 2
   positional arguments but 3 were given

```

Output matches the worked trace in the assignment exactly, including both `TypeError` messages from the illegal calls.

Submission Note

Test 4 called `parse_payload` with `raw_packet` and `delimiter` as keywords, which isn't allowed since both sit before the `/` in the signature. Python raised `"parse_payload() got some positional-only arguments passed as keyword arguments: 'raw_packet, delimiter'"`. Test 5 tried to pass `correction_offset` positionally as a third argument, which isn't allowed since it sits after the `*`. Python raised `"parse_payload() takes 2 positional arguments but 3 were given"`. Both matched what the assignment predicted.